

schengen: Supplementary Material

Anonymous Authors
Anonymous Institution(s)

This document collects the supplementary material for `schengen`. It is referenced from the main paper and released with the artifact. Section numbering is independent of the main paper.

APPENDIX A SCHENGEN: USER INTERFACE (UI) AND AVAILABLE CONSTRAINTS

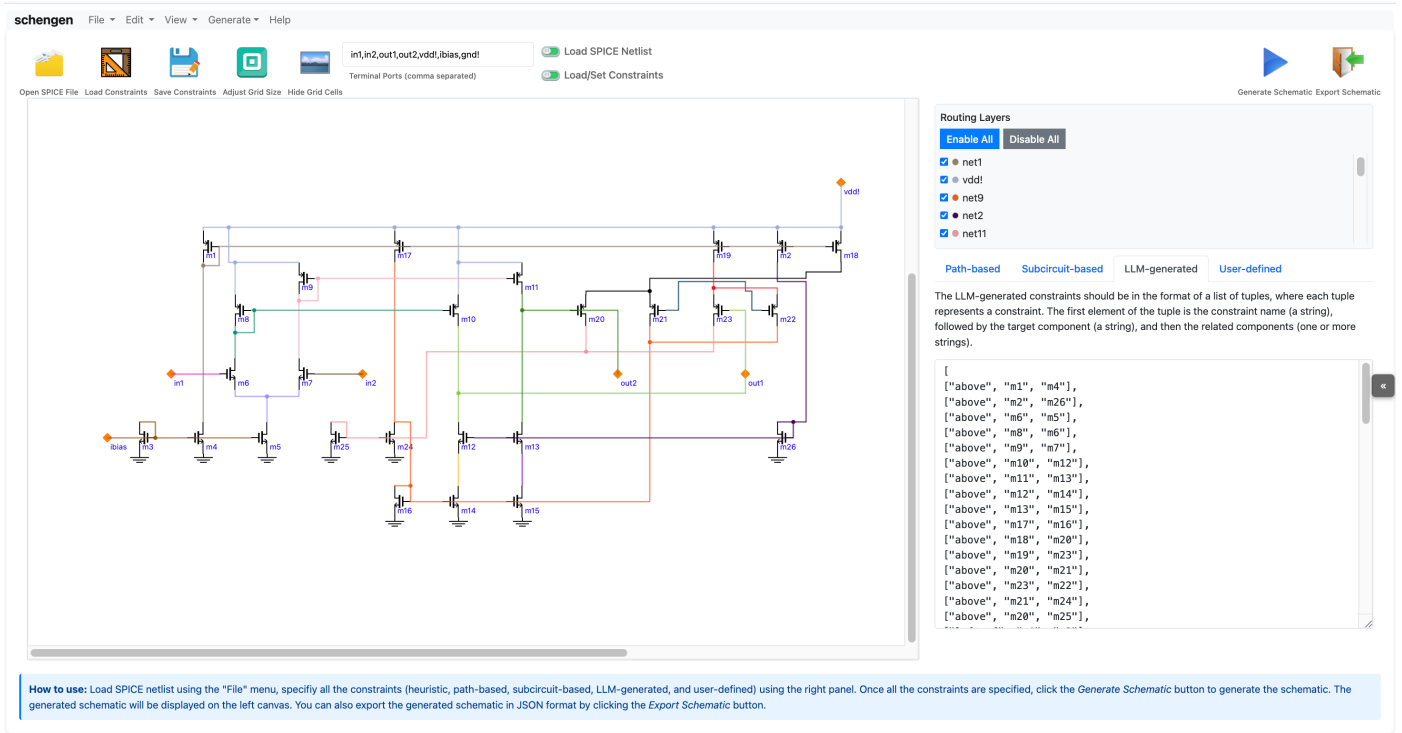


Fig. 1. The Graphical User Interface (GUI) of `schengen`.

TABLE I
AVAILABLE CONSTRAINTS SUPPORTED BY SCHENGEN. THESE CONSTRAINTS SERVE AS PRIMITIVE OPERATORS FOR BOTH LLMs AND USERS TO CONSTRUCT CONSTRAINT SETS THAT DETERMINE COMPONENT PLACEMENT ON A GRID-BASED CANVAS. THE VISUALIZATION OF THESE CONSTRAINTS IS SHOWN IN FIG. 3.

#	Constraint	Description
1	["above", "A", "B"]	A is above B
2	["below", "A", "B"]	A is below B
3	["left_of", "A", "B"]	A is on the left of B
4	["right_of", "A", "B"]	A is on the right of B
5	["same_row", "A", "B"]	A and B share the same row
6	["same_column", "A", "B"]	A and B share the same column
7	["same_column_above", "A", "B"]	A and B share the same column, A is above B
8	["same_column_below", "A", "B"]	A and B share the same column, A is below B
9	["between_row", "A", "B", "C", ..., "X"]	A should be placed between the minimum and maximum rows of B, C, ..., X (the column can be different)
10	["between_column", "A", "B", "C", ..., "X"]	A should be placed between the minimum and maximum columns of B, C, ..., X (the row can be different)
11	["clear_row_between", "A", "B"]	Keep rows between A and B clear (no component in same column between their rows)
12	["clear_column_between", "A", "B"]	Keep columns between A and B clear (no component in same row between their columns)
13	["row_proximity", n, "A", "B"]	The row of A is close to B (within n cell distance)
14	["column_proximity", n, "A", "B"]	The column of A is close to B (within n cell distance)
15	["cluster_proximity", n, "A", "B", "...", "X"]	Keep A, B, ..., X close to each other (row and column), within n cell distance
16	["t_junction", "A", "B", "C"]	Keep B and C in the same row and within three columns of each other, with A centered at the midpoint column between them, in a nearby distinct row (the stem of the T). Order-agnostic in B and C

APPENDIX B SUBCIRCUIT LAYOUT TEMPLATES AND CONSTRAINT VISUALIZATION

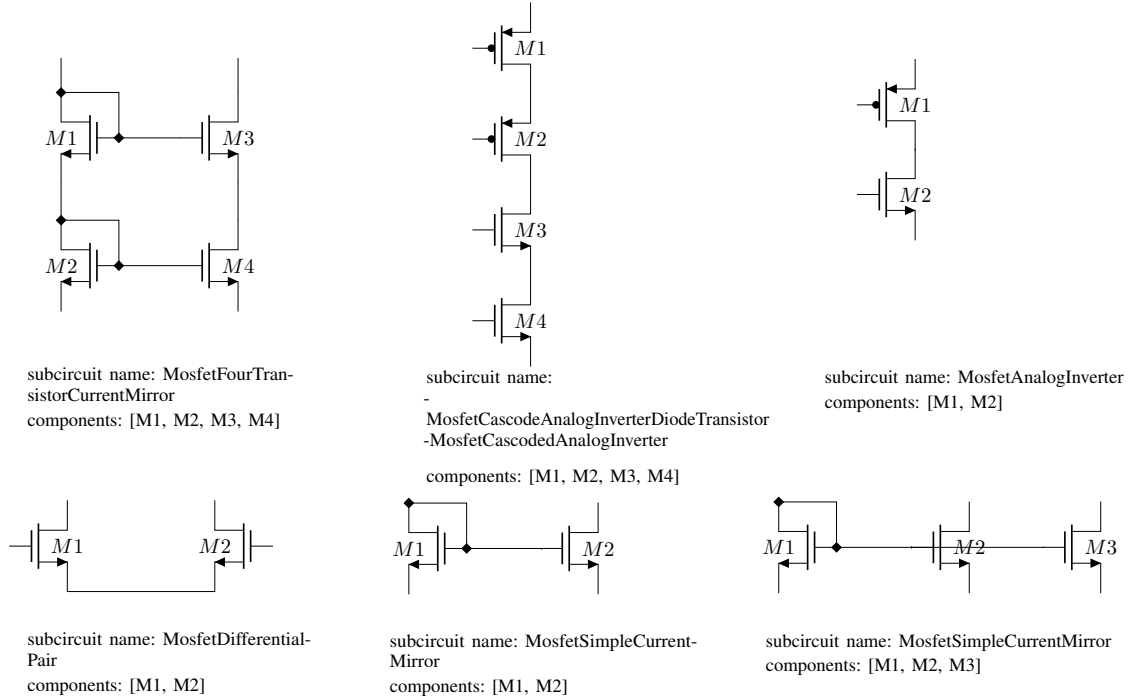


Fig. 2. Available subcircuit layout templates supported by schengen. These templates can be used to impose placement constraints on groups of transistors. The ordering of components is important, as it determines how constraints are applied between devices. In the schengen GUI, a subcircuit constraint can be specified using the subcircuit name together with an ordered list of components. For example, {"MosfetDifferentialPair[1]": ["m6", "m7"]} enforces proximity and same-row placement constraints between m6 and m7. The index [1] serves as a unique subcircuit identifier to distinguish multiple instances of subcircuits with the same name.

APPENDIX C IMPLEMENTATION DETAILS

Algorithm 1 gives the placement procedure referenced from the main paper.

Prompt construction: The instruction template used for constraint generation is compact (roughly 150 lines, versioned and released with the artifact), while the prompt actually sent is two to four times longer, depending on circuit size, because the

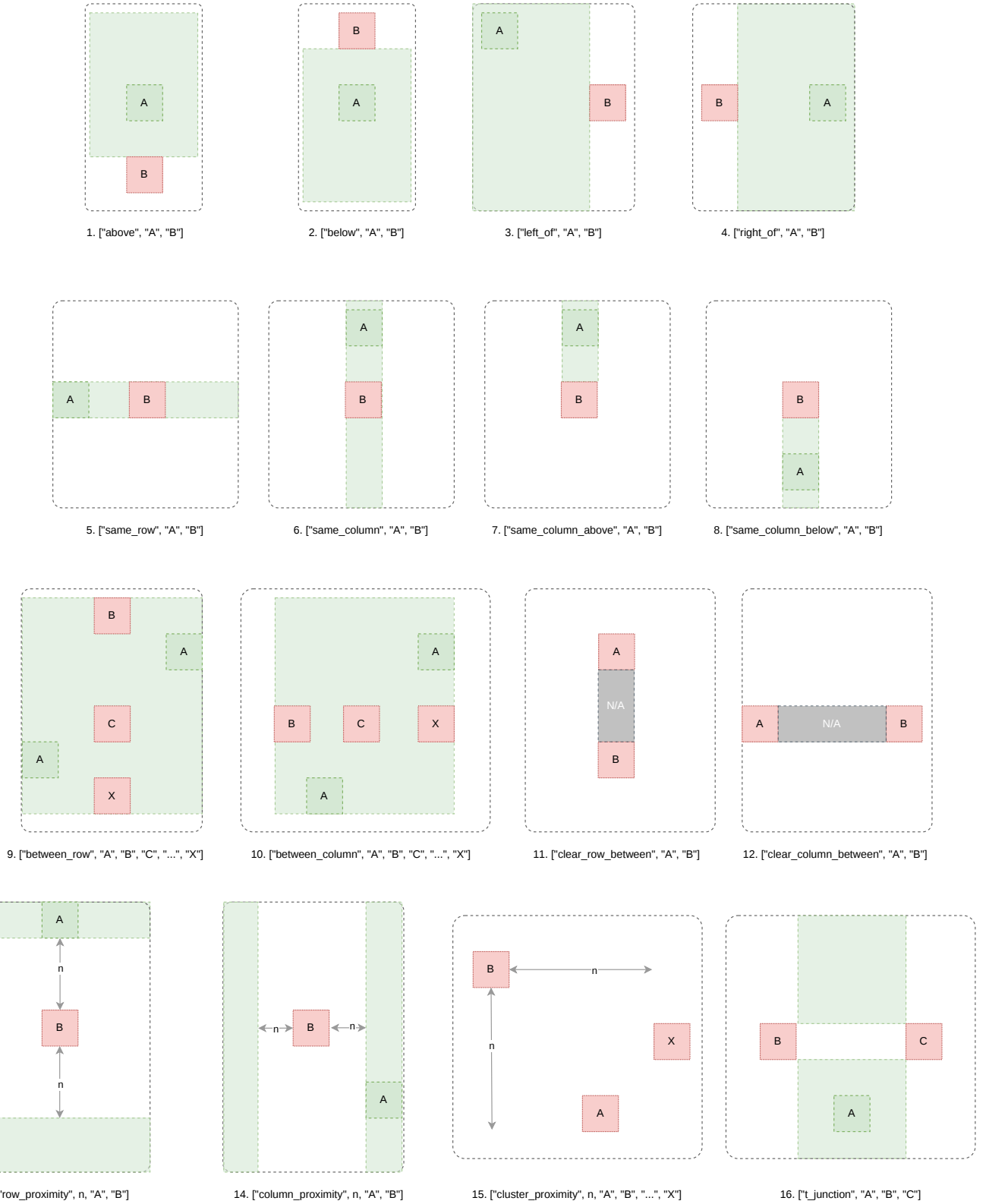


Fig. 3. Visualization of the constraints supported by *schengen*. Reference components are illustrated by red boxes, while target components are illustrated by green boxes. The **green** regions indicate valid placement locations for target components, whereas the **gray** regions denote forbidden placement areas where no components may be placed.

Algorithm 1 Constraint-based Placement in `schengen`

Require: Netlist N , LLM constraints L , user constraints U , constraint programming (CP) model m

Ensure: Placement and orientation solution S

```
1:  $C \leftarrow \text{ANALYZECIRCUIT}(N) \cup \text{RECOGNIZEBUILDINGBLOCKS}(N)$ 
2:  $C \leftarrow C \cup U$ 
3: function SOLVETH(C)
4:    $T \leftarrow \text{TRANSLATECONSTRAINTS}(C)$  // OR-Tools constraints.
5:    $H \leftarrow \text{GETSCHEMATICSIZE}(N, C) + \text{HPWLCOST}(N, C)$ 
6:    $m.\text{SETCONSTRAINTS}(T)$ 
7:    $m.\text{MINIMIZE}(\text{ESTIMATEROUTINGCOST}(N, T) + H)$ 
8:   return  $m.\text{SOLVE}()$ 
9: end function
10: for all  $l \in L$  do ▷ Add LLM constraints (sequentially)
11:   if SOLVETH( $C \cup \{l\}$ ) then
12:      $C \leftarrow C \cup \{l\}$ 
13:   end if
14: end for
15:  $S \leftarrow \text{SOLVETH}(C)$ 
16: return  $S$ 
```

SPICE netlist and the constraint-primitive vocabulary are substituted automatically. The vocabulary is rendered from the same registry the solver uses, so the prompt cannot offer the model a primitive that the solver does not implement, and because the only per-circuit substitution is the netlist itself, the template cannot be tuned circuit by circuit.

Runtime breakdown: Constraint admission accounts for a median of 71% of total runtime when LLM-generated constraints are present, and routing for only a few seconds even on the largest circuits. Because admission cost scales with the number of proposed constraints, the GPT-5.4 configuration is roughly $1.7\times$ slower than the Opus 4.8 one while producing slightly worse schematics.

Efficient constraint admission: The placement algorithm of the main paper specifies *which* constraints are admitted. The implementation realizes the same admitted set more efficiently, by first testing the entire batch for joint feasibility and bisecting only around the offending constraints when it fails. This reduces the number of solver invocations from one per constraint to $O(k \log n)$ for k conflicting constraints, and the feasibility checks discard the placement-quality objective since only their yes/no answer is used. The two formulations are equivalent in output and differ by more than an order of magnitude in runtime on large constraint sets.

APPENDIX D

SOFT ADMISSION OF LLM-GENERATED CONSTRAINTS

The prioritization scheme of the main paper treats admitted constraints as hard: a constraint is either enforced exactly or dropped. `schengen` additionally supports applying LLM-generated constraints as weighted objective penalties, so that the solver selects a maximum-weight satisfiable subset jointly with the placement objective rather than committing to a drop/keep decision in advance. This recovers constraints that a sequential admission order discards: when two LLM constraints conflict, softening both allows the solver to choose between them in the context of the full objective. The trade-off is that the resulting model carries one indicator variable per soft constraint, which optimizes less effectively on large circuits under a fixed solver budget, and that the solver trades wirelength for structure, increasing HPWL. Hard admission is therefore the default, and all results reported in this paper use it. The soft variant is available for circuits where satisfying the maximum number of structural relations matters more than wirelength.

APPENDIX E

PER-CIRCUIT ROUTING QUALITY ANALYSIS

This section expands the aggregate routing-quality results of the main paper.

Table II lists the per-circuit numbers that the aggregate results and every subgroup claim of the main paper are computed from. Its Total row reproduces the crossing counts of Table I of the main paper.

Crossings: Crossings improve on 12 of the 15 circuits, are unchanged on 1 and regress on 2. The size of the improvement is *not* a function of circuit size: circuits below 25 devices improve by 33% on average and circuits at or above 25 devices by 31%. Two circuits explain the exceptions. On example-4 the entire LLM constraint set is rejected during admission because it conflicts with the accepted subcircuit groups, so that circuit’s LLM configurations are identical to the baseline by construction. On example-3, the largest circuit and the only bipolar one, all three configurations sit within half a crossing of one another. It has no subcircuit information and the fewest generated constraints, and appears to be dominated by routing congestion that placement cannot relieve.

HPWL: HPWL improves on 12 circuits and regresses on 3, and the magnitude of the improvement grows sharply with size: circuits below 25 devices improve by 6% on average, while those at or above 25 devices improve by 20%. The mechanism is the one anticipated in earlier, smaller-scale experiments: LLM-generated constraints tend to place related components together and

TABLE II

PER-CIRCUIT DATA BEHIND THE SUBGROUP CLAIMS OF THE MAIN PAPER, FROM THE SAME $3 \text{ ARMS} \times 15 \text{ CIRCUITS} \times 10 \text{ SEEDS}$ RUNS THAT TABLE I OF THE MAIN PAPER AGGREGATES. CROSSINGS ARE PER-CIRCUIT MEANS OVER SUCCESSFUL SEEDS. SUB. MARKS THE FIVE CIRCUITS FOR WHICH ACST RECOGNIZES AT LEAST ONE GROUP. PROP./ADM. ARE THE CONSTRAINTS PROPOSED BY THE MODEL AND ADMITTED AFTER CONFLICT RESOLUTION.

			#crossings			Opus 4.8		GPT-5.4		
Ex	Dev	Sub.	w/o	Opus	GPT	Prop.	Adm.	Prop.	Adm.	
1	26	✓	24.9	22.7	19.0	58	46	101	78	
2	30	✓	11.4	13.0	17.1	65	50	116	85	
3	48		213.2	213.8	212.6	28	23	84	40	
4	20	✓	6.0	6.0	5.0	39	0	65	0	
5	14	✓	4.0	3.0	4.0	30	26	42	32	
6	36	✓	79.4	35.9	59.5	64	60	118	103	
7	19		9.3	2.9	3.3	27	23	46	38	
8	17		15.1	12.1	16.5	33	30	56	49	
9	19		13.1	7.7	10.4	30	28	66	56	
10	44		113.2	78.9	81.8	68	63	111	98	
11	25		34.4	15.4	18.1	44	38	75	61	
12	27		29.8	9.2	14.6	40	33	62	47	
13	18		8.4	4.5	4.0	34	30	67	52	
14	35		73.2	41.8	47.0	59	44	72	62	
15	24		26.0	19.0	8.5	40	35	85	69	
Total			5	661	486	521	659	529	1166	870

others slightly farther apart, trading wirelength for the routing space that removes bends and crossings. On a small circuit there is little congestion to relieve and the additional spacing is close to a net cost. Once a circuit is large enough for congestion to dominate, the same trade pays for itself several times over. The three regressions are correspondingly small, at most 10%, while the largest gains reach 30–34%.

Generator verbosity: GPT-5.4 is substantially more verbose than Opus 4.8. Verbosity is not free: admission time scales with the number of proposed constraints, which is why the GPT-5.4 configuration is roughly $1.7\times$ slower. It can also cost quality directly: on example-6, GPT-5.4 admits 103 constraints against Opus 4.8’s 60 and produces 60 crossings against 36, the largest per-circuit gap between the two.

Per-circuit differences between generators: Neither model dominates circuit by circuit (GPT-5.4 is markedly better on example-15, Opus 4.8 on example-12, example-9 and example-6), but Opus 4.8 is better in aggregate.

APPENDIX F ADDITIONAL FIGURES

Fig. 4 shows the constraint generation prompt and Fig. 5 the placement constraints associated with a four-transistor current mirror. Fig. 6 shows the Example 3 schematics with and without LLM-generated constraints.

Constraint Generation Prompt

Act as a senior analog IC designer. Given a SPICE netlist {{NETLIST}}, building blocks ({{STRUCTURE_RESULT}}), and optional partitioning ({{PARTITIONING_RESULT}}), generate a placement constraint list using only primitives from {{CONSTRAINT_PRIMITIVES}}.

Objectives:

- Create a human-readable, grid-based schematic.
- Reflect signal flow (left-to-right) and bias flow (top-to-bottom).
- Align symmetric/matched devices and group functional blocks.

Strategy:

Order constraints from relaxed (e.g., ``right_of``) to strict (e.g., ``same_row``). Ensure strict constraints do not contradict earlier ones.

Rules:

- Use exact, case-sensitive component names from {{NETLIST}}
- No redundant/conflicting constraints or new components
- Output only the final Python-style list assignment; no explanations or reasoning.
- Devices belonging to the same building block are spatially grouped
- If partitioning is provided, stages are visually separated
- Format: `["constraint-name", value, "target", "ref"]`. Omit the value field entirely if the primitive does not require one.

Deliverable:

```
[  
  ["<name>", "<target-component>", "<related-components>"],  
  ["<name>", "<target-component>", "<related-components>"],  
  ["<name>", value, "<target-component>", "<related-components>"],  
  ...  
]
```

Fig. 4. Constraint generation prompt used in `schengen`, shown in simplified form. The constraint-primitive vocabulary and the netlist are substituted automatically, from the solver's primitive registry and from the input circuit respectively. The full template and the exact prompt sent for every benchmark circuit are released with the artifact.

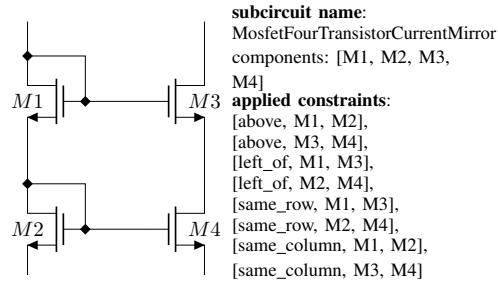


Fig. 5. Placement constraints for a four-transistor current mirror, ensuring that the transistors are in the correct order.

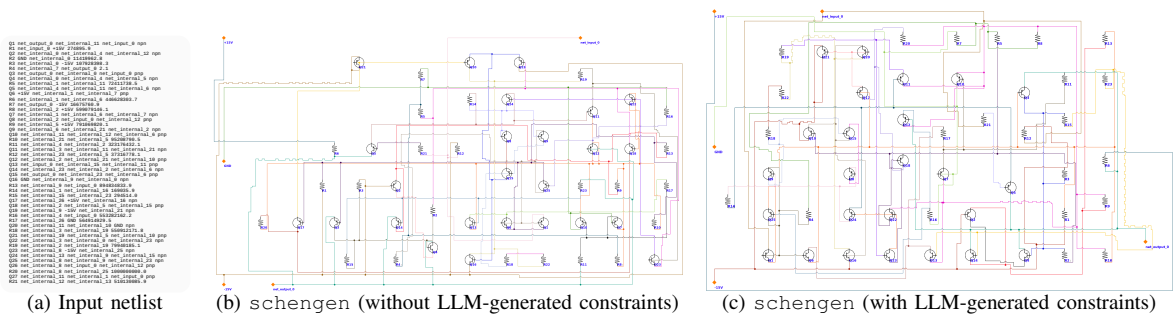
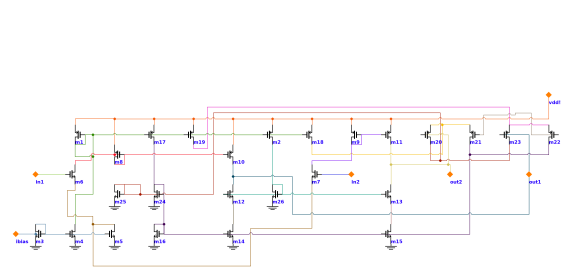
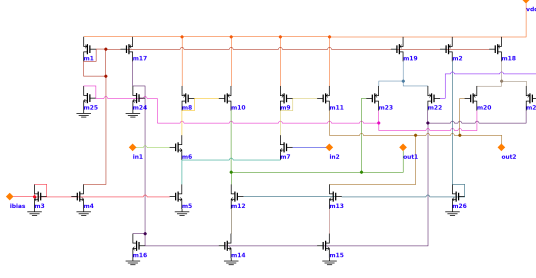


Fig. 6. (Example 3) Comparison of schematics generated by schengen with and without LLM-generated constraints.

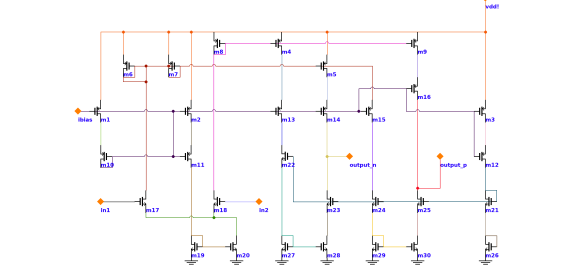
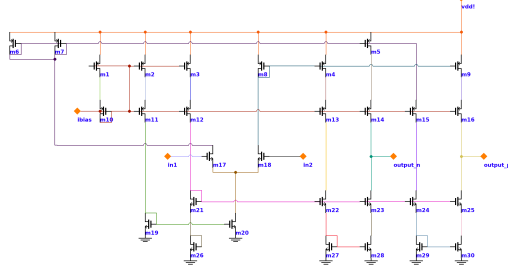
with LLM-generated constraints

without

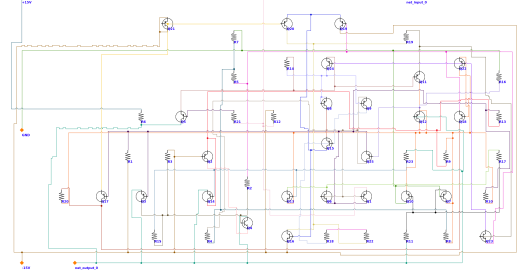
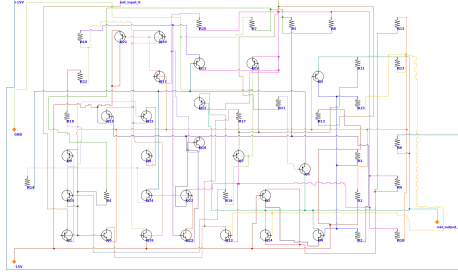
example-1
Two-stage single-ended
MOS OTA



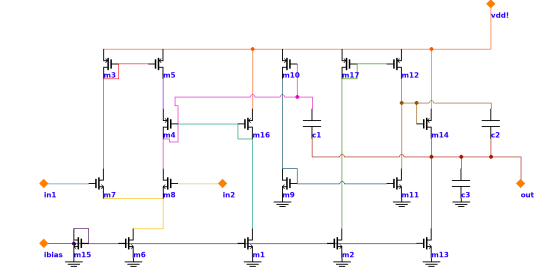
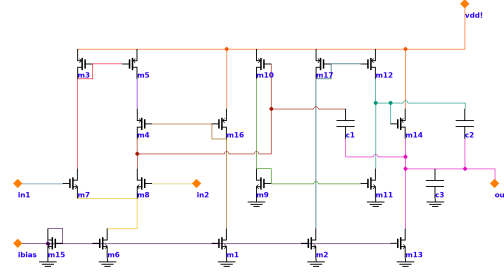
example-2
Single-stage folded-cascode
OTA



example-3
BJT cube-root function
circuit



example-4
Two-stage cascode
MOS OTA



example-5
Two-stage cascode
single-ended MOS OTA

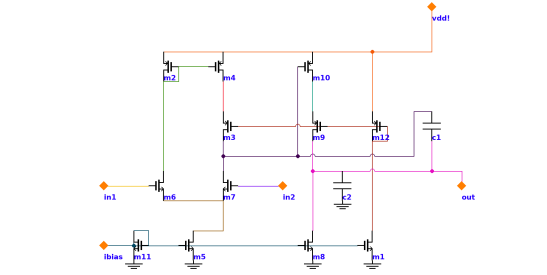
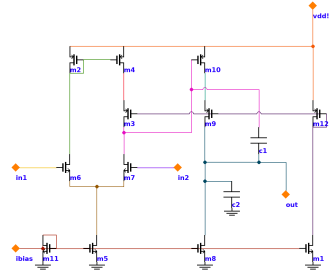
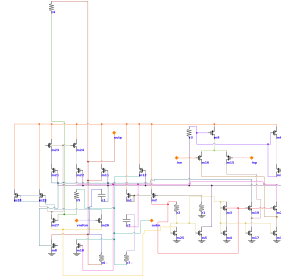
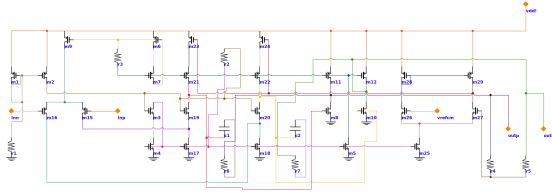


Fig. 7. Schematics generated by schengen for all 15 benchmark circuits, with (left) and without (right) LLM-generated placement constraints. Both columns use the identical base constraint set (subcircuit, path-based, orientation and user-defined constraints are the same), so the two differ only in whether the LLM-generated list is included. Renders use a single fixed seed and are representative rather than best-of. Exact geometry varies with seed, but the structural difference is consistent with the averaged metrics reported in the main paper. Circuits 1–5 are shown here, with 6–15 following.

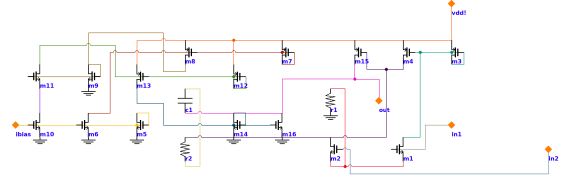
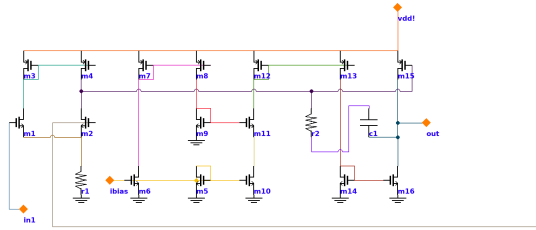
with LLM-generated constraints

without

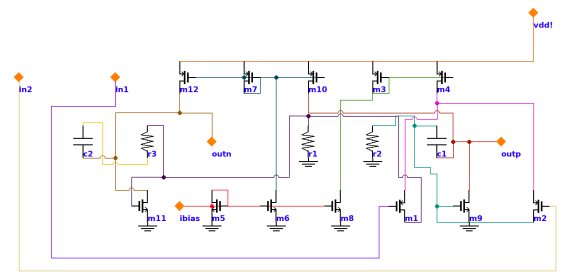
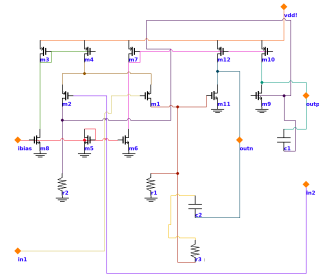
example-6
Single-stage
fully-differential
folded-cascode OTA



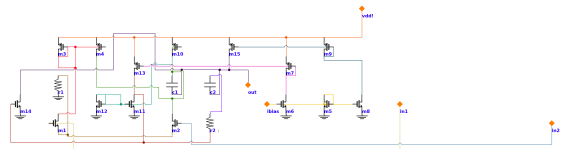
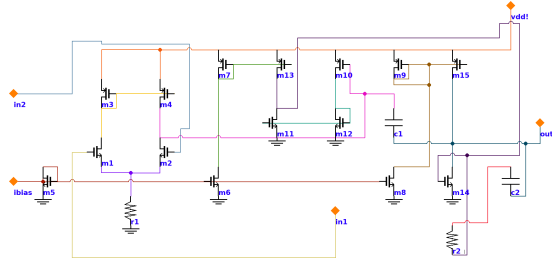
example-7
Two-stage single-ended
MOS op-amp



example-8
Two-stage fully-differential
MOS op-amp



example-9
Two-stage single-ended
MOS op-amp



example-10
Two-stage fully-differential
MOS op-amp with CMFB

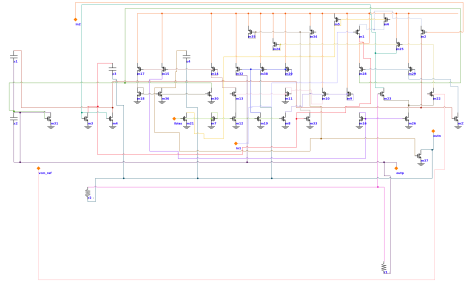
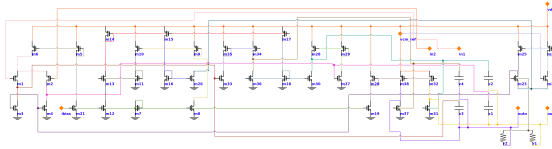
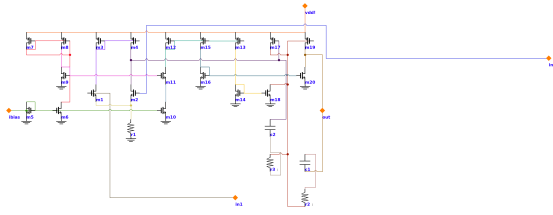
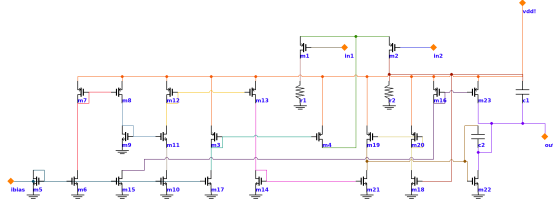


Fig. 8. Generated schematics with (left) and without (right) LLM-generated placement constraints, circuits 6–10. See Fig. 7 for details.

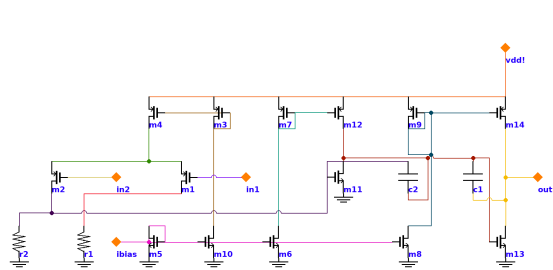
with LLM-generated constraints



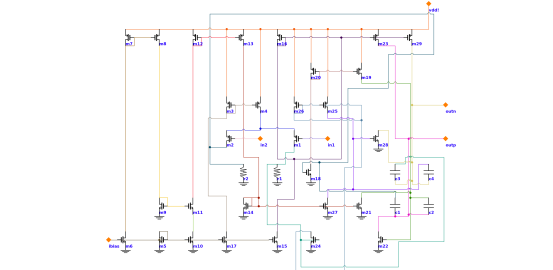
example-11
Two-stage single-ended
MOS op-amp



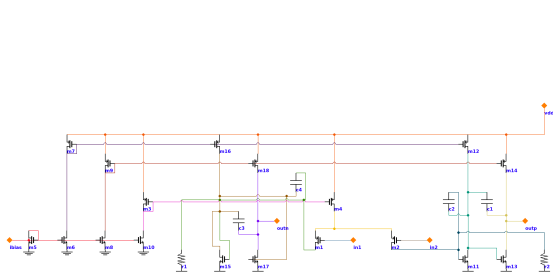
example-12
Three-stage NMC
single-ended MOS op-amp



example-13
Three-stage RNMC
single-ended MOS op-amp



example-14
Three-stage NMC
fully-differential MOS
op-amp



example-15
Three-stage RNMC
fully-differential MOS
op-amp

without

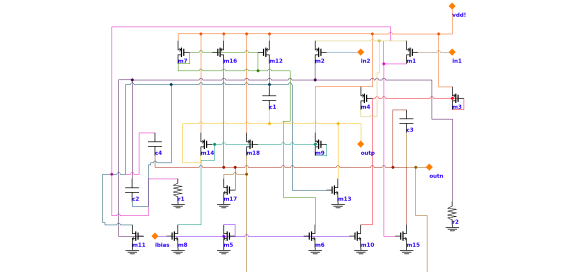
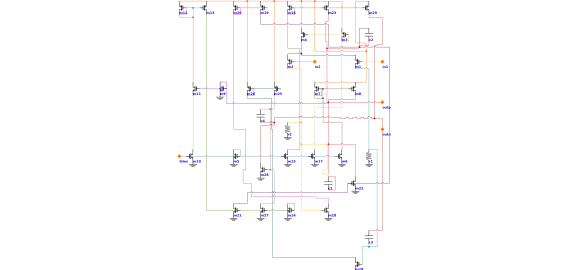
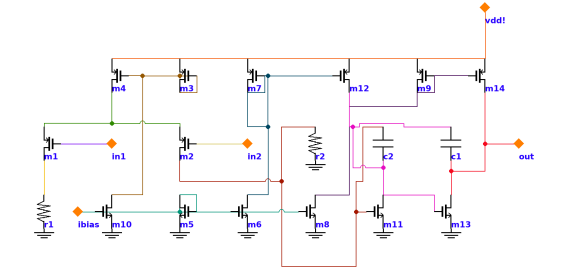
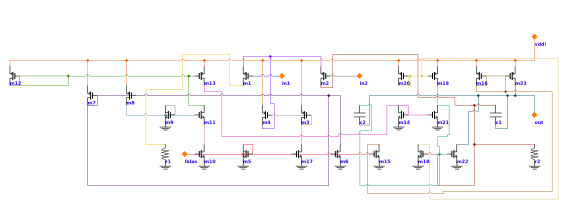
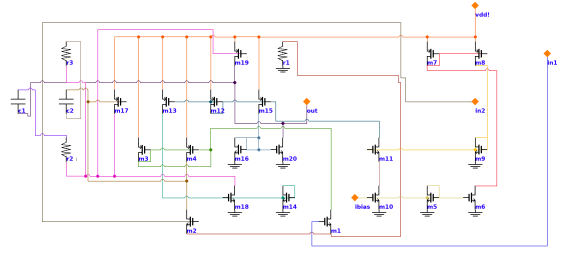


Fig. 9. Generated schematics with (left) and without (right) LLM-generated placement constraints, circuits 11–15. See Fig. 7 for details.